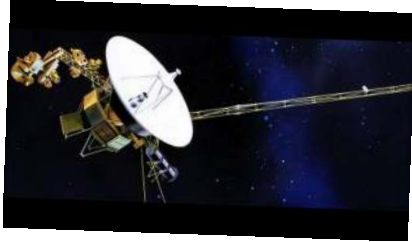


_r su _ _ o e _ WTBERIS

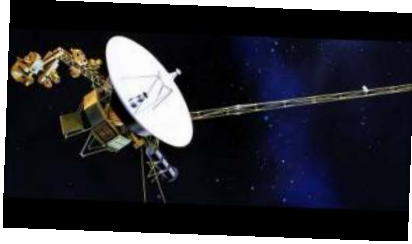
_av _ _ m _ c _ E4DBLK

_a _ c _ 2 _ _ 6 NUHSY

These are items



Some send data wirelessly



Voyager Probe



QR Code



Cell Phones



Some store data



Compact Disk

Hard Drive

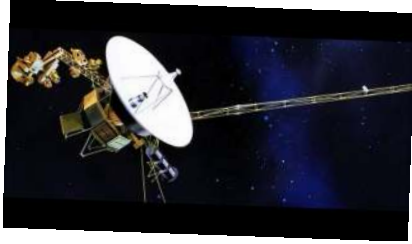


QR Code



Flight Recorder

They have **problems**



Cosmic Radiation



Scratches



Magnetic Interference



Dirt, Damage, Bad Pics



Interference

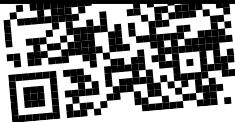


...

They have **problems**



Somehow, they all work anyway

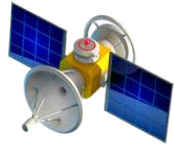


Dirt, Damage, Bad Pics

Interference

...

I (voyager probe) want to send 1000 numbers back to Earth

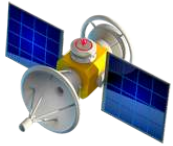


Voyager 1



Earth

I (voyager probe) want to send 1000 numbers back to Earth



Voyager 1

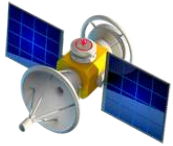
6 7 4 2 0 ... 6 9



Earth

I (voyager probe) want to send 1000 numbers back to Earth

There is a 99% chance each number actually makes it



Voyager 1

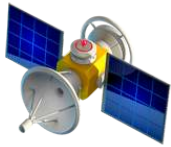
6 ? 4 2 ? ... ? 9



Earth

I (voyager probe) want to send 1000 numbers back to Earth

There is a 99% chance each number actually makes it



Voyager 1

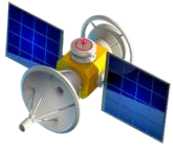
6 ? 4 2 ? ... ? 9



Earth

$$99\% \times 99\% \times \dots \times 99\% = (99\%)^{1000} = 0.004\%$$

What if I send everything twice?



Voyager 1

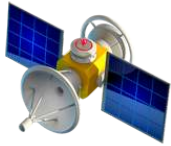
6 6 7 7 4 4 2 2 0 0 ... 6 6 9 9



Earth

What if I send everything twice?

Some are still lost



Voyager 1

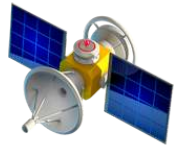
6 6 ? 7 ? ? 2 2 ? 0 ... ? 6 9 ?



Earth

What if I send everything twice?

Some are still lost



Voyager 1

6 6 ? 7 ? ? 2 2 ? 0 ... ? 6 9 ?



Earth

$$99.99\% \times \dots \times 99.99\% = (99.99\%)^{1000} \approx 90\%$$

Voyager's messages are actually closer to 1,200,000 numbers long

Voyager's messages are actually closer to 1,200,000 numbers long

$$(99.99\%)^{1200000} \approx 0\%$$

Voyager's messages are actually closer to 1,200,000 numbers long

$$(99.99\%)^{1200000} \approx 0\%$$

Have to use 4 copies to get to 90%

Voyager's messages are actually closer to 1,200,000 numbers long

$$(99.99\%)^{1200000} \approx 0\%$$

Have to use 4 copies to get to 90%

Lots of overhead, would take Voyager 28 minutes to send

Voyager's messages are actually closer to 1,200,000 numbers long

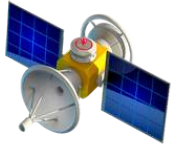
$$(99.99\%)^{1200000} \approx 0\%$$

Have to use 4 copies to get to 90%

Lots of overhead, would take Voyager 28 minutes to send

Need a better approach...

“Parity” Messages



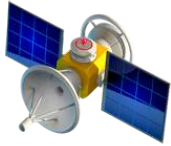
Voyager 1



Earth

“Parity” Messages

Send the data, as usual, ...



Voyager 1

6 7

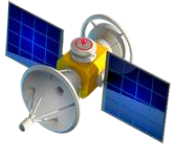
...



Earth

“Parity” Messages

Send the data, as usual, **then send some extra stuff**



Voyager 1

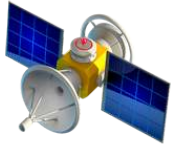
6 7 (6+7) ...



Earth

“Parity” Messages

Send the data, as usual, **then send some extra stuff**



Voyager 1

6 7 (6+7) ...



Earth

Use algebra to find the missing value:

x 7 13

$$\mathbf{x} = 13 - 7 = 6$$

6 **y** 13

$$\mathbf{y} = 13 - 6 = 7$$

Reed-Solomon Code (Irving Reed and Gustave Solomon, 1960)

Reed-Solomon Code (Irving Reed and Gustave Solomon, 1960)

Send $N + M$ total numbers:

- N are “data” numbers
- M are “parity” numbers (from equations)

Reed-Solomon Code (Irving Reed and Gustave Solomon, 1960)

Send $N + M$ total numbers:

- N are “data” numbers
- M are “parity” numbers (from equations)

Any N numbers are enough to decode the original message

Reed-Solomon Code (Irving Reed and Gustave Solomon, 1960)

Send $N + M$ total numbers:

- N are “data” numbers
- M are “parity” numbers (from equations)

Any N numbers are enough to decode the original message

Voyager uses “chunks” of 255 numbers. 233 data + 22 parity.

Reed-Solomon Code (Irving Reed and Gustave Solomon, 1960)

Send $N + M$ total numbers:

- N are “data” numbers
- M are “parity” numbers (from equations)

Any N numbers are enough to decode the original message

Voyager uses “chunks” of 255 numbers. 233 data + 22 parity.

$\approx$ 100% chance of message delivery, **only 10% overhead**

Problem

Computer numbers have a fixed range.

Problem

Computer numbers have a fixed range.

A “1 byte” number can have a value in the range 0-255.

Problem

Computer numbers have a fixed range.

A “1 byte” number can have a value in the range 0-255.

If an equation “overflows” the number, we’re in trouble.

Problem

Computer numbers have a fixed range.

A “1 byte” number can have a value in the range 0-255.

If an equation “overflows” the number, we’re in trouble.

Example

$$200 + 200 = 400$$

400 is too big!

Math is made up

Math is made up

“Numbers” aren’t real; they are just “things” that we do stuff to

Math is made up

“Numbers” aren’t real; they are just “things” that we do stuff to

“+” is made up; it takes in two things and gives back one thing

Math is made up

“Numbers” aren’t real; they are just “things” that we do stuff to

“+” is made up; it takes in two things and gives back one thing

We have a problem with +

Math is made up

“Numbers” aren’t real; they are just “things” that we do stuff to

“+” is made up; it takes in two things and gives back one thing

We have a problem with +

Can we just change what it does?

Clock Math



Clock Math

A clock face never “overflows”



Clock Math

A clock face never “overflows”

$$10 + 3 = 1$$



Clock Math

A clock face never “overflows”

$$10 + 3 = 1$$

And has a **finite** number of things (1, 2, ..., 12).



Clock Math

A clock face never “overflows”

$$10 + 3 = 1$$

And has a **finite** number of things (1, 2, ..., 12).

Just change $+$ to work like a clock



Clock Math

A clock face never “overflows”

$$10 + 3 = 1$$

And has a **finite** number of things $(1, 2, \dots, 12)$.

Just change $+$ to work like a clock

This guy proved that “math still works” sometimes



Évariste Galois
1811 – 1832

Clock Math

A clock face never “overflows”

$$10 + 3 = 1$$

And has a **finite** number of things $(1, 2, \dots, 12)$.

Just change $+$ to work like a clock

This guy proved that “math still works” sometimes

Then died in a duel



Évariste Galois
1811 – 1832

Finite Field / Galois Field Rules

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**
- OR, the number of things is prime^2 , prime^3 , ...

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**
- OR, the number of things is prime^2 , prime^3 , ...
- $+$ and $\times$ follow a few rules

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**
- OR, the number of things is prime^2 , prime^3 , ...
- $+$ and $\times$ follow a few rules

2 is prime.

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**
- OR, the number of things is prime^2 , prime^3 , ...
- $+$ and $\times$ follow a few rules

2 is prime. Computers love powers of 2.

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**
- OR, the number of things is prime^2 , prime^3 , ...
- $+$ and $\times$ follow a few rules

2 is prime. Computers love powers of 2. **This is luck**

Finite Field / Galois Field Rules

Math only works when:

- The number of things is **prime**
- OR, the number of things is prime^2 , prime^3 , ...
- $+$ and $\times$ follow a few rules

2 is prime. Computers love powers of 2. **This is luck**

Use Galois fields to solve the overflow problem

_r su _ _ o e _ WTBERIS

_av _ _ m _ c _ E4DBLK

_a _ c _ 2 _ _ 6 NUHSY

?r?su?? ?o?e? WTBERIS

?av?? ?m?c? E4DBLK

?a?c? 2??6 NUHSY

Erasure Codes **WTBERIS**

David Zmick **E4DBLK**

March 2026 **NUHSY**

Appendices

Galois Field Size Rules

Must have either:

- A prime number of elements
- p^n elements, for some prime p and some integer n

Galois Field Operation Rules

Addition:

- Associativity: $(a + b) + c = a + (b + c)$
- Commutativity: $a + b = b + a$
- Identity: An identity (0) exists where: $a + 0 = a$
- Inverses: for each a there is some b such that $a + b = 0$

Multiplication:

- Commutativity: $a \times b = b \times a$
- Associativity: $(a \times b) \times c = a \times (b \times c)$
- Identity : A multiplicative identity (1) where: $a \times 1 = a$
- Inverses: for each $a \neq 0$ there is some b such that $a \times b = 1$

Both:

- Distributivity: $a \times (b + c) = (a \times b) + (a \times c)$

Galois Field for 2 elements

2 is prime.

Elements: $\{0, 1\}$.

$$\begin{array}{r|rr} + & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 0 \end{array}$$

$$\begin{array}{r|rr} + & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 0 \end{array}$$

Can check by hand the all properties work

Can also do this where count is prime: 3, 5, 7, 11, etc

$$\begin{array}{r|rrr} + & 0 & 1 & 2 \\ \hline 0 & 0 & 1 & 2 \\ 1 & 1 & 2 & 0 \\ 2 & 2 & 0 & 1 \end{array}$$

“Extension fields” are of count p^n where p is prime.

Galois Field Extension Fields

To create field $\text{GF}(p^n)$, create first $\text{GF}(p)$.

Field elements are polynomials over some x , with $c_i \in \text{GF}(p)$:

$$c_0x^0 + c_1x^1 + \dots + c_nx^n$$

Can add:

$$(a_0x^0 + \dots + a_nx^n) + (b_0x^0 + \dots + b_nx^n) = (a_0 + b_0)x^0 + \dots + (a_n + b_n)x^n$$

Can multiply (foil method, or something called convolution):

$$\begin{aligned}(a_0 + a_1x) \times (b_0 + b_1x) &= (a_0b_0) + (a_1xb_0) + (a_0b_1x) + (a_1xb_1x) \\ &= a_0b_0 + (a_1b_0 + a_0b_1)x + (a_1b_1)x^2\end{aligned}$$

Galois Field Construction (cont..)

Divide polynomials using “long division” from elementary school

Leaves a remainder

To construct the field:

- Pick a polynomial $R(x)$ where $R(x) \neq 0$ for any x
- Addition: add polynomials, then result divide by R . Keep only the remainder
- Multiplication derived from addition

GF(2²) over $x^2 + x + 1$

+	0	1	x	$x + 1$
0	0	1	x	$x + 1$
1	1	0	$x + 1$	x
x	x	$x + 1$	0	1
$x + 1$	$x + 1$	x	1	0

$\times$	0	1	x	$x + 1$
0	0	0	0	0
1	0	1	x	$x + 1$
x	0	x	$x + 1$	1
$x + 1$	0	$x + 1$	1	x

Reed-Solomon Equations

Message is: $M_0M_1\dots M_{N-1}$.

Where M_i are elements of some Galois Field $\text{GF}(N)$.

Create a polynomial over x with coefficients in $\text{GF}(N)$:

$$f(x) = M_0x^0 + M_1x^1 + M_2x^2\dots + M_{N-1}x^{N-1}$$

To generate $K = N + M$ total words, send:

$$f(0)f(1)\dots f(K-1)$$

Use linear algebra to correct errors or erasures at receiver.